

Project 7 — Python Bioinformatics Pipeline for Protein FASTA Analysis

Research question: Can a short Python program automate basic protein-sequence quality checks and summary statistics?

This project is the first automation-focused project in the portfolio. Instead of performing every sequence calculation manually, a small Python script reads FASTA records, calculates sequence statistics, scans for selected residues, and writes a simple summary table.

Why this matters

Bioinformatics workflows often repeat the same operations across many sequences. Automation reduces manual work, makes the analysis reproducible, and creates a bridge between biological questions and programming.

Python bioinformatics pipeline workflow

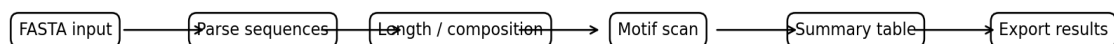


Figure 1. Original workflow diagram for the pipeline.

Pipeline design

- Input: one or more protein sequences in FASTA format.
- Parsing: read each header and concatenate sequence lines.
- Statistics: calculate sequence length, counts of selected amino acids and approximate composition.
- Motif scan: search for user-defined short patterns.
- Output: produce a compact tabular report that can be reused in downstream analysis.

Core Python implementation

```
from pathlib import Path

def read_fasta(path):
    records = []
    name, seq = None, []
    for line in Path(path).read_text().splitlines():
        line = line.strip()
        if not line:
            continue
        if line.startswith(">"):
            if name is not None:
                records.append((name, "".join(seq)))
            name = line[1:]
            seq = []
        else:
            seq.append(line.upper())
    if name is not None:
        records.append((name, "".join(seq)))
    return records

def summarize(name, seq):
    return {
        "id": name,
        "length": len(seq),
        "A": seq.count("A"),
        "G": seq.count("G"),
```

```

        "C": seq.count("C"),
        "K": seq.count("K"),
        "R": seq.count("R"),
    }

records = read_fasta("lysozyme.fasta")
results = [summarize(name, seq) for name, seq in records]

for row in results:
    print(row)

```

How the code works

- `read_fasta()` separates headers from sequence lines and returns clean sequence strings.
- `summarize()` calculates simple features without requiring a specialised bioinformatics package.
- The same functions can be reused for dozens or hundreds of FASTA records.
- The design intentionally starts with standard Python so that the logic is understandable before introducing Biopython.

Example output structure

Sequence ID	Length	A	G	C	K	R
lysozyme_1	129	...	...	...	...	...
lysozyme_2	129	...	...	...	...	...
lysozyme_3	129	...	...	...	...	...

The ellipses above indicate example output fields rather than invented sequence-specific counts. When the pipeline is run with the actual FASTA file, the cells are populated automatically.

Biological interpretation

The pipeline does not replace biological reasoning. Its value is that it makes repetitive calculations consistent and auditable. For a protein family such as lysozyme, the same program can quickly compare sequence length and composition across homologs before a researcher performs more specialised analyses such as alignment, motif analysis or phylogenetics.

What I learned

This project connected programming with a real bioinformatics workflow. I learned how FASTA files are structured, how sequence data can be represented as strings, how small functions can automate repetitive analysis, and why reproducible code is useful in computational biology.

Limitations

- The pipeline performs basic statistics rather than full sequence alignment.
- It does not calculate evolutionary distances or construct a phylogenetic tree.
- Motif detection is exact-string based and does not account for substitutions unless explicitly programmed.
- A production workflow would add input validation, error handling and richer output formats.

Sources

- NCBI FASTA/Protein documentation and sequence records.
- RCSB Protein Data Bank for structural context.
- Python standard library concepts used for FASTA parsing and string analysis.